

Bases de Datos II

SEGURIDAD Y TRANSACCIONES



INTRODUCCIÓN

- La seguridad de las bases de datos se refiere a la protección frente a **accesos malintencionados**
- Es posible controlar el acceso a la base de datos brindando la autorización adecuada
- Los datos guardados en la base de datos deben estar protegidos contra:
 - accesos no autorizados
 - destrucción o alteración malintencionadas
 - introducción accidental de inconsistencias

Seguridad en Base de Datos

Tipos de Amenazas



- **Pérdida de integridad.** La integridad se pierde si se realizan cambios no autorizados en los datos mediante acciones intencionadas o accidentales
- **Pérdida de disponibilidad.** Se refiere a que los objetos estén disponibles para un usuario humano o para un programa que tenga los derechos correspondientes
- **Pérdida de confidencialidad.** La confidencialidad de la base de datos tiene relación con la protección de los datos frente al acceso no autorizado

Seguridad en Base de Datos

Niveles de Seguridad

- **Sistema de bases de datos.** Autorización al acceso a una parte limitada de la base de datos
- **Sistema operativo.** La debilidad de la seguridad del sistema operativo puede servir como medio para el acceso no autorizado
- **Red.** La seguridad en el nivel del software de la red es tan importante en Internet y en las redes privadas de las empresas
- **Físico.** Los sitios que contienen los sistemas informáticos deben estar protegidos de intrusos
- **Humano.** Los usuarios deben ser autorizados cuidadosamente para reducir la posibilidad de intrusión



Seguridad en Base de Datos

Autenticación y Autorización

- La **autenticación** es el proceso por el cual se identifica un usuario como válida para posteriormente acceder a ciertos recursos definidos
 - Relacionado con la *gestión de usuarios y control de acceso* al DBMS
- La **autorización** es el proceso sobre el cual se establecen qué tipos de recursos están permitidos o denegados para cierto usuario o grupo de usuarios concreto
 - Relacionado con *permisos* (lectura, escritura) para cada usuario autenticado

Seguridad en Base de Datos

Cifrado de Datos - Conceptos

- El cifrado de datos se utiliza para proteger datos confidenciales como los números de las tarjetas de crédito y contraseñas
- El cifrado se puede utilizar también para proporcionar protección adicional a partes confidenciales de la base de datos
- Los datos se codifican utilizando algún algoritmo de codificación o cifrado
- Un usuario no autorizado que acceda a datos codificados tendrá dificultades para descifrarlos, pero a los usuarios autorizados se les proporcionarán claves para descifrar los datos

Seguridad en Base de Datos

Mecanismos de Seguridad (MySQL)

- Administración de cuentas de usuarios
 - Creación, modificación y borrado de cuentas de usuario
- Gestión de permisos
 - Otorgamiento, Modificación y Revocación de privilegios
- Conexiones seguras
 - Tipos de conexiones y soporte para SSL

Mecanismos de Seguridad

Administración de Usuarios

- Un usuario MySQL se define en términos de un nombre de usuario y una contraseña/password
- Por defecto, el motor de base de datos crea un usuario con permisos para todas las tablas de la base de datos
 - El superusuario se denomina ROOT
 - Se recomienda limitar el uso de ROOT a la gestión del DBMS y no usarlo en aplicaciones de producción

Mecanismos de Seguridad

Creación de Usuarios

- **CREATE USER** 'nombre_usuario'@'host' **IDENTIFIED BY** 'tu_contraseña';
- Seguido al nombre de usuario, se debe especificar la IP desde donde podrá realizar conexiones a la base de datos el usuario creado. Puede ser:
 - 'localhost' o '127.0.0.1', desde la misma PC en la que se encuentre instalado MySQL, es decir el host local
 - '192.168.1.100', sólo permite conexiones desde dicha IP (utilizada para identificar a un PC en un LAN)
 - '%' es un comodín que permite conexiones desde cualquier IP

Mecanismos de Seguridad

Concesión de Privilegios a Usuarios

- **GRANT** [**permiso**] **ON** [nombre de bases de datos]. [nombre de tabla] **TO** '[usuarioX]'@'host' [**WITH GRANT OPTION**];
- **Tipos de Permisos**
 - **ALL** : esta opción otorga todos los permisos
 - **CREATE**: permite crear nuevas tablas o bases de datos
 - **DROP**: permite eliminar tablas o bases de datos
 - **DELETE**: permite eliminar registros de tablas
 - **INSERT**: permite insertar registros en tablas
 - **SELECT**: permite leer registros en las tablas
 - **UPDATE**: permite actualizar registros seleccionados en tablas
 - **WITH GRANT OPTION**: permite que **usuarioX** maneje privilegios de otros usuarios



Roles

- `CREATE USER 'dev1'@'localhost' IDENTIFIED BY 'dev1pass';`
- `CREATE ROLE 'app_developer';`
- `DROP ROLE 'app_developer';`
- `GRANT 'app_developer' TO 'dev1'@'localhost';`
- `SHOW GRANTS FOR 'dev1'@'localhost' USING 'app_developer';`
- `GRANT SELECT ON T1 to 'app_developer';`

Mecanismos de Seguridad

Revocar Privilegios a Usuarios

- **REVOKE** [permisos] **ON** [nombre de base de datos] [nombre de tabla] **FROM** '[nombre de usuario]' @'host';

Una vez que se haya finalizado con la configuración de privilegios (GRANT o REVOKE) se deben refrescar todos los con el comando:

- **FLUSH PRIVILEGES;**

Mecanismos de Seguridad

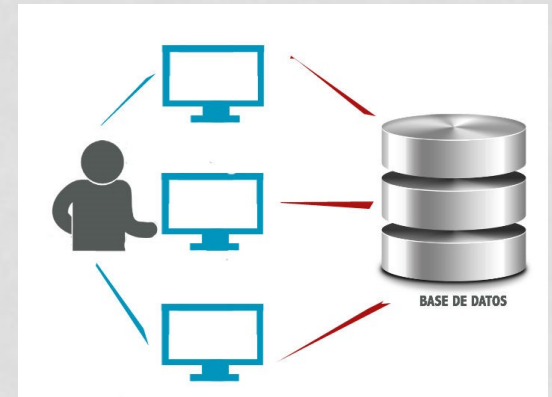
Privilegios a Usuarios - Ejemplos

- **GRANT ALL ON** midb.usuarios **TO** 'juan'@'%'
WITH GRANT OPTION;
- **GRANT ALL ON** midb.* **TO** 'admin'@'%';
- **GRANT SELECT ON** midb.log **TO**
'auditor'@'192.168.1.100';
- **REVOKE ALL ON** midb.* **FROM** 'admin'@'%';
- **FLUSH PRIVILEGES;**

Transacciones en Base de Datos

Conceptos

- Una transacción es un mecanismo para definir las unidades lógicas del procesamiento de una base de datos
- Una transacción se inicia por la ejecución de un programa escrito en un lenguaje de programación
- Una transacción está delimitada por instrucciones de la forma *inicio transacción* y *fin transacción*



Transacciones en Base de Datos

Fallos

- Fallo de la computadora (caída del sistema)
- Un error de la transacción o del sistema
- Errores locales o condiciones de excepción detectados por la transacción
- Control de la concurrencia
- Fallo del disco rígido
- Problemas físicos y catástrofes



Transacciones en Base de Datos

ACID (AC--)

Atomicidad

- Requiere que cada transacción sea "todo o nada": si una parte de la transacción falla, todas las operaciones de la transacción fallan, y por lo tanto la base de datos no sufre cambios

Consistencia

- Asegura que cualquier transacción llevará a la base de datos de un estado válido a otro estado válido. Cualquier dato que se escriba en la base de datos tiene que ser válido de acuerdo a todas las reglas definidas

Transacciones en Base de Datos

ACID (--ID)

Aislamiento

- Asegura que la ejecución concurrente de las transacciones resulte en un estado del sistema que se obtendría si estas transacciones fueran ejecutadas una detrás de otra

Durabilidad

- Significa que una vez que se confirmó una transacción quedará persistida, incluso ante eventos como pérdida de alimentación eléctrica, errores y caídas del sistema

Transacciones en Base de Datos

Estados

- **Activa**, la transacción permanece en este estado durante su ejecución
- **Parcialmente confirmada**, después de ejecutarse la última instrucción
- **Fallida**, tras descubrir que no puede continuar la ejecución normal
- **Abortada**, después de haber retrocedido la transacción y restablecido la base de datos a su estado anterior al comienzo de la transacción
- **Confirmada**, tras completarse con éxito

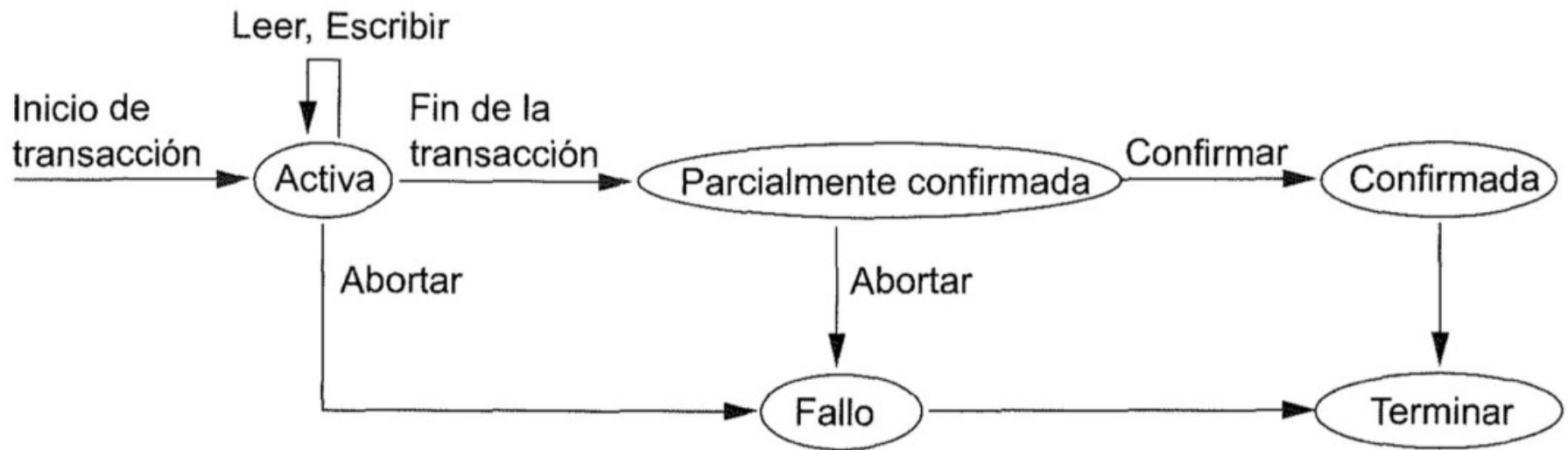
Transacciones en Base de Datos

Operaciones

- **INICIO DE TRANSACION.** Marca el inicio de la ejecución de una transacción
- **LEER o ESCRIBIR.** Especifican operaciones de lectura o escritura en los elementos de la base de datos que se ejecutan como parte de una transacción
- **FIN DE LA TRANSACION.** Especifica que las operaciones LEER y ESCRIBIR de la transacción han terminado y marca el final de la ejecución de la transacción. En este punto se comprueba si los cambios introducidos pueden confirmarse
- **CONFIRMAR (Commit).** Señala una finalización satisfactoria de la transacción, por lo que los cambios (actualizaciones) ejecutados se pueden enviar con seguridad a la base de datos
- **ABORTAR (Rollback).** Señala que la transacción no ha terminado satisfactoriamente, por lo que deben deshacerse los cambios

Transacciones en Base de Datos

Diagrama de Transición de Estados



Transacciones en Base de Datos

Concurrencia de transacciones

- **Enfoque de ejecución secuencial** de transacciones es más sencillo de implementar, pero menos eficiente. Para comenzar una transacción es necesario finalizar la anterior
- **Enfoque de ejecución concurrente** permite varias transacciones que actualizan concurrentemente los datos y puede provocar complicaciones en la consistencia de los mismos

El **esquema de control de concurrencia** de un DBMS controla la interacción entre las transacciones concurrentes para evitar que se destruya la consistencia de la base de datos

Transacciones en Base de Datos

Ventajas de la Concurrency

- **Productividad y utilización de recursos mejorados**
 - El número de transacciones que puede ejecutar en un tiempo dado aumenta cuando varias transacciones se ejecutan en paralelo. Por ej., las operaciones de E/S, como uso de CPU y discos pueden trabajar en paralelo en una computadora
- **Tiempo de espera reducido**
 - Frente a transacciones cortas y largas, la ejecución concurrente reduce los retardos impredecibles en la ejecución de las transacciones
 - Se reduce también el tiempo medio de respuesta

Problemas de Concurrency

- Debido a la ejecución simultánea de transacciones, pueden surgir problemas como:
- **Race conditions:** Situación donde el resultado de las transacciones depende del orden de ejecución. Por ejemplo, dos transacciones intentando actualizar el mismo saldo de cuenta simultáneamente.
- **Dirty Read:** Ocurre cuando una transacción lee datos modificados por otra transacción que aún no ha sido confirmada. Si la segunda transacción se deshace, los datos leídos son inválidos. Por ejemplo, un cliente ve un saldo de cuenta que se actualizó pero luego se revertió, dejando al cliente con información incorrecta.

Problemas de Concurrency

- **Non-repeatable Read:** Se produce cuando una transacción lee un dato y luego otra transacción modifica ese dato antes de que la primera transacción se complete. Esto significa que si la primera transacción lee el mismo dato de nuevo, obtendrá un valor diferente. Por ejemplo, una transacción que lee un precio de producto, y otra transacción cambia el precio antes de que la primera confirme su operación.
- **Phantom Read:** Sucede cuando una transacción lee un conjunto de datos y otra transacción inserta o elimina datos en ese conjunto antes de que la primera transacción termine. Esto resulta en un conjunto de datos que cambia durante la transacción. Por ejemplo, una transacción que cuenta el número de registros que cumplen ciertos criterios, mientras que otra transacción inserta nuevos registros que también cumplen esos criterios.

Mecanismos de Control de Concurrency

Locking

Consiste en controlar el acceso a los datos mediante bloqueos:

- **Shared Lock:** Permite que múltiples transacciones lean un dato, pero ninguna pueda modificarlo hasta que se libere el bloqueo. Ej: varias transacciones pueden leer el saldo de una cuenta simultáneamente, pero no pueden modificarlo hasta que se libere el bloqueo.
- **Exclusive Lock:** Permite que una sola transacción lea y modifique el dato. Nadie más puede leer ni modificar el dato hasta que se libere el bloqueo. Ej: una transacción que actualiza el saldo de una cuenta bloquea el dato para que otros usuarios no puedan leer ni modificarlo.

Mecanismos de Control de Concurrency

Control de Versiones (Optimistic Concurrency Control)

- En el **control de versiones** las transacciones no utilizan bloqueos. En lugar de eso, se permite que las transacciones realicen cambios y, antes de confirmarlas, se verifica si los datos han cambiado durante la transacción.
- **Ejemplo:** Una transacción que actualiza un registro debe comparar el valor del registro antes de la actualización con el valor actual en la base de datos. Si ha cambiado, se produce un conflicto y la transacción debe manejar el error.

Mecanismos de Control de Concurrency

Timestamp Ordering

- En el **timestamp ordering**, cada transacción recibe un sello de tiempo y los datos se ordenan según estos tiempos. Las transacciones se ejecutan en el orden de sus sellos de tiempo para evitar conflictos.
- **Ejemplo:** Si dos transacciones modifican el mismo registro, la transacción con el sello de tiempo más bajo tiene prioridad.

Niveles de Aislamiento

Los **niveles de aislamiento** determinan el grado en el que una transacción debe estar aislada de las demás:

- **Read Uncommitted:** Permite lecturas de datos no confirmados (dirty reads). Proporciona el mayor rendimiento pero la menor consistencia.
- **Read Committed:** Permite leer solo datos confirmados. Minimiza los dirty reads pero no evita las lecturas no repetibles.
- **Repeatable Read:** Garantiza que si una transacción lee un dato, ese dato no cambiará durante la transacción. Evita lecturas no repetibles pero no previene las lecturas fantasma.
- **Serializable:** Proporciona el mayor nivel de aislamiento, tratando las transacciones como si fueran serializadas. Esto asegura la consistencia más alta pero puede impactar en el rendimiento.

Ejemplos prácticos

-- Ejemplo de bloqueo en SQL Server

```
BEGIN TRANSACTION;
```

```
SELECT * FROM productos WITH (UPDLOCK);
```

-- Bloqueo de actualización

```
UPDATE productos SET stock = stock - 1 WHERE id = 1;
```

```
COMMIT TRANSACTION;
```

Ejemplos prácticos

-- Ejemplo de control de versiones en PostgreSQL

BEGIN;

SELECT * FROM productos FOR UPDATE; -- Bloqueo de actualización

UPDATE productos SET stock = stock - 1 WHERE id = 1;
COMMIT;

Índices

- Estructuras complementarias en DBMS que se asocian con los atributos de las tablas y agilizan la operaciones de búsqueda
- Juegan el mismo papel que los índices de los libros o los catálogos de fichas de las bibliotecas
- Por ejemplo, para recuperar un registro de cuenta dado su número de cuenta, el sistema de bases de datos buscaría en un índice para encontrar el bloque de disco en que se encuentra el registro correspondiente, y entonces extraería ese bloque de disco para obtener el registro cuenta



Transacciones en Base de Datos

Tipos de Índices

- **Índices ordenados.** Estos índices están basados en una disposición ordenada de los valores. Ej: B-trees.
- **Índices asociativos.** Estos índices están basados en una distribución uniforme de los valores a través de una serie de cajones (buckets). El valor asignado a cada cajón está determinado por una función, llamada función de asociación (hash function). Ej: Hashes.

Transacciones en Base de Datos

Índices Ordenados: Subtipos

- **Índice primario o índices con agrupación.** La clave de búsqueda especifica el orden secuencial del archivo (archivos ordenados secuencialmente). La clave de búsqueda de un índice primario es normalmente la clave primaria (pero no necesariamente)
- **Índices secundarios o índices sin agrupación.** Las claves de búsqueda especifican un orden diferente del orden secuencial del archivo. Un archivo puede tener varios índices secundarios además de su método de acceso principal

Transacciones en Base de Datos

Índices Multinivel

- Solución para índices muy grandes
- El índice se trata como si fuese un archivo secuencial y se construye otro índice sobre el índice con agrupación
- Para localizar un registro se usa en primer lugar una búsqueda binaria sobre el índice más externo para buscar el registro con el mayor valor de la clave de búsqueda que sea menor o igual al valor deseado
- Los índices multinivel están estrechamente relacionados con la estructura de árbol, tales como los árboles binarios usados para la indexación en memoria

Transacciones en Base de Datos

Índices en SQL

create index <nombre-índice> **on** <nombre-tabla>
(<lista-atributos>)

drop index <nombre-índice>

lista-atributos es la lista de atributos de la tabla que constituye la clave de búsqueda del índice

Ejemplo:

create index índice-s **on** sucursal (**nombre-sucursal**)

índice llamado índice-s de la tabla sucursal con la clave de búsqueda nombre-sucursal

Comparación de índices

B-tree vs. Hash

- Se puede utilizar un índice de árbol B para comparaciones de columnas en expresiones que utilizan los operadores =, >, >=, <, <= o BETWEEN.
- El índice también se puede utilizar para comparaciones LIKE si el argumento de LIKE es una cadena constante que no comienza con un carácter comodín.
- For example, the following statements use indexes:

```
SELECT * FROM tbl_name WHERE key_col LIKE 'Patrick%';  
SELECT * FROM tbl_name WHERE key_col LIKE 'Pat%_ck%';
```
- The following statements do not use indexes:

```
SELECT * FROM tbl_name WHERE key_col LIKE '%Patrick%';  
SELECT * FROM tbl_name WHERE key_col LIKE other_col;
```

Comparación de índices

B-tree vs. Hash

- Las **Hashes** se utilizan únicamente para comparaciones de igualdad que utilizan los operadores $=$ o $\leq$ (pero son muy rápidos). No se utilizan para operadores de comparación como $<$ que encuentran un rango de valores.
- El optimizador no puede utilizar un índice hash para acelerar las operaciones ORDER BY.
- Solo se pueden utilizar **claves completas** para buscar una fila.

Ejemplo en MySQL

- CREATE INDEX MYINDEX ON USERS (DNI) **USING
HASH**;